Techniques for Optimizing Deep Learning on FPGA

Reduced Precision (Fixed-Point and Binarization)

Sparsh Mittal

Acronyms

- ALM = adaptive logic module
- FPGA – Field-programmable gate array
- FxP – Fixed point
- FP – Floating point
- DSP – Digital signal processor
- FF – Flip-flop
- SIMD – Single instruction multiple data
- NN – Neural network
- BW – Bandwidth
- MAC – Multiplier accumulator
- CUDA – Compute Unified Device Architecture
- LUT – Lookup table

Using FxP Format for FPGA Implementation

- Recent FPGAs have many DSPs, which implement hard multiplier, preadder, and accumulator core.
- Compared to 32-b FP, FxP with narrow bitwidth reduces resource consumption.

	Xilinx Logic				Xilinx DSP			Altera DSP	
	multiplier		adder		multiply & add			multiply & add	
	LUT	FF	LUT	FF	LUT	FF	DSP	ALM	DSP
fp32	708	858	430	749	800	1284	2	1	1
fp16	221	303	211	337	451	686	1	213	1
fixed32	1112	1143	32	32	111	64	4	64	3
fixed16	289	301	16	16	0	0	1	0	1
fixed8	75	80	8	8	0	0	1	0	1
fixed4	17	20	4	4	0	0	1	0	1

Guo, et al. "A survey of FPGA-based neural network accelerator." arXiv preprint arXiv:1712.08934 (2017)

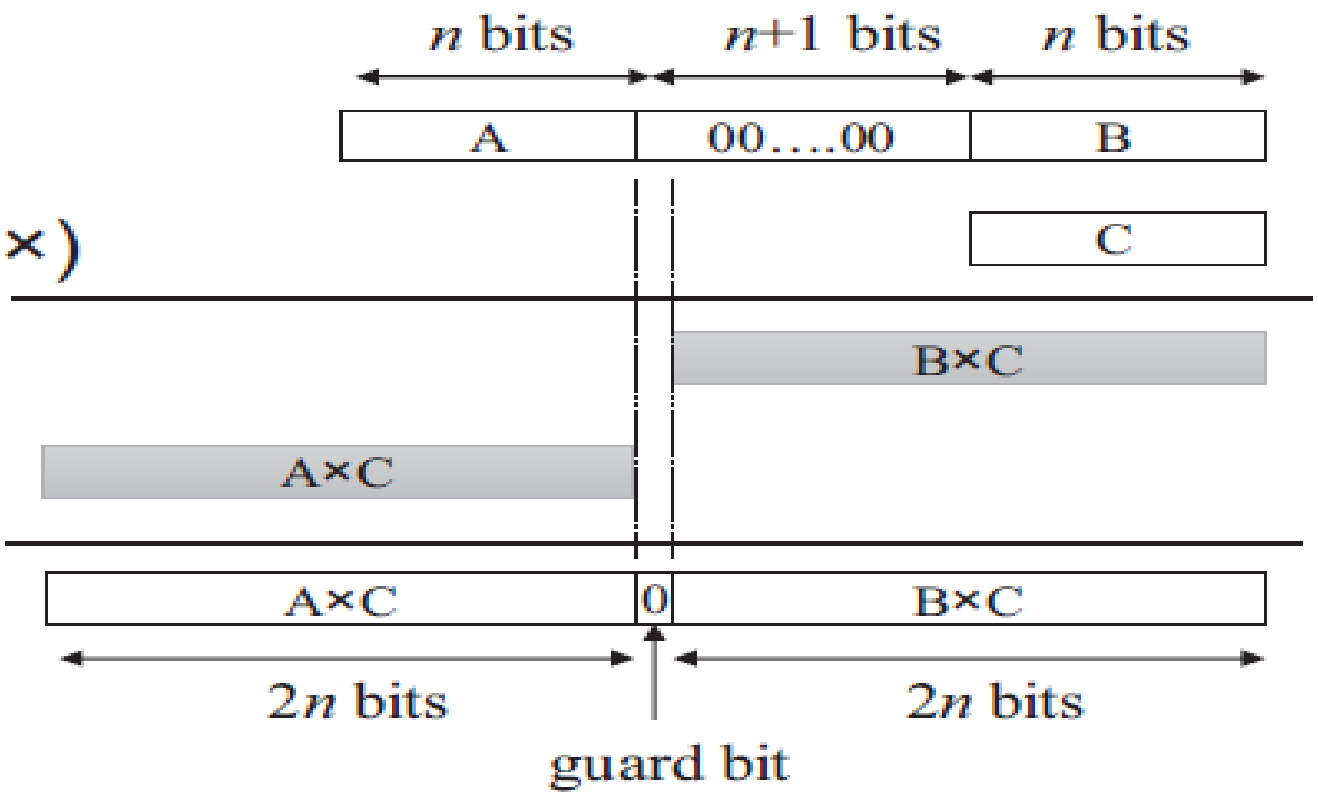
Doing Concurrent FxP Multiplications

- FxP with ≤ 16 b data can fit into 1 DSP unit.

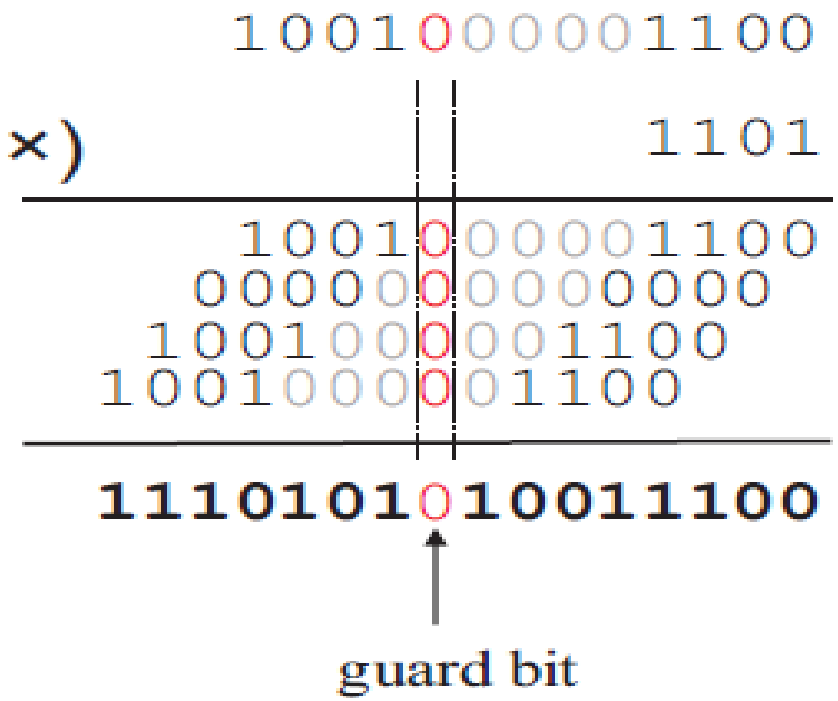
⇒ No extra benefit of using 4 b or 8 b. Wide multipliers and adders remain underutilized.

- Idea: implement two narrow FxP multiplication with a single wide FxP multiplication.
- Execute AC and BC as $C(A << k+B)$.
- If k is very large, the output of BC and AC do not overlap and can be easily separated.
- For example, two 8-b multiplications are implemented with a 25×18 multiplier and k=9.

Reference: D. Nguyen, D. Kim, and J. Lee, "Double MAC: Doubling the performance of convolutional neural networks on modern FPGAs," in *Proceeding of the Design, Automation & Test in Europe Conference & Exhibition*, 2017, pp. 890–893.



(a) n-bit SIMD unsigned multiplier



(b) Example: 9×13 & 12×13

To perform two n-bit multiplications in a SIMD fashion, we need one $(3n+1)\times n$-bit multiplier, e.g., with a 25×18-b multiplier, n can be at most 8

Reference: D. Nguyen, D. Kim, and J. Lee, "Double MAC: Doubling the performance of convolutional neural networks on modern FPGAs," in *Proceeding of the Design, Automation & Test in Europe Conference & Exhibition*, 2017, pp. 890–893.

Binarized NN (BNN)

Binarized NN (BNN)

- Instead of 32-b FP or 8 b/16 b FxP values, BNNs use 1-b values ⇒ lower memory capacity and BW needs.
- Binarized data have higher cacheability ⇒ attractive because FPGA has low memory capacity.
- For example, 8-b FxP AlexNet requires 50 MB.
- Binarized AlexNet requires 7.4 MB memory.

⇒ Binarized AlexNet can be fully stored in on-chip RAM of FPGA.

- On a typical FPGA, an FxP NN becomes bound by memory BW, but BNN nearly reaches the peak of its computational performance.

Optimizing BNNs on FPGA (1/2)

$$\begin{bmatrix} -1 & +1 & +1 \end{bmatrix} \otimes \begin{bmatrix} -1 & -1 & +1 \\ +1 & -1 & +1 \\ +1 & -1 & +1 \end{bmatrix} = \begin{bmatrix} (-1.-1) \\ +(+1.+1) \\ +(+1.+1) \end{bmatrix} \begin{bmatrix} (-1.-1) \\ +(+1.-1) \\ +(+1.-1) \end{bmatrix} \begin{bmatrix} (-1.+1) \\ +(+1.+1) \\ +(+1.+1) \end{bmatrix} = \begin{bmatrix} 3 & -1 & +1 \end{bmatrix}$$

(a) An example of binarized MM

- BNNs allow converting MAC operations into XNOR and population count (aka bit count or BCNT) operations.
- Instead of DSP blocks, XNOR gates can be used.

$$\begin{bmatrix} 0 & 1 & 1 \end{bmatrix} \otimes \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 1 \\ 1 & 0 & 1 \end{bmatrix} = \begin{pmatrix} \text{BCNT(XNOR(011,011))} \\ \text{BCNT(XNOR(011,000))} \\ \text{BCNT(XNOR(011,111))} \end{pmatrix}^T = \begin{bmatrix} 3 & -1 & +1 \end{bmatrix}$$

(b) Binarized MM using XNOR and BCNT. -1 is represented using 0.

Optimizing BNNs on FPGA (2/2)

BCNT= OneCount-ZeroCount

IN	Computation	OUT
000	-1-1-1= -3	101
001	-1-1+1 = -1	111
010	-1+1-1= -1	111
011	-1+1+1= +1	001
100	+1-1-1= -1	111
101	+1-1+1= +1	001
110	+1+1-1= +1	001
111	+1+1+1= +3	011

(c) BCNT using a lookup table (OUT is in 2's complement form)

Bit count can be realized using an LUT

Large reduction in area/energy!

Advantages of BNN on FPGA

- FPGAs provide native support for bitwise operations.

⇒ BNNs can be implemented efficiently.

- BNNs can allow FPGAs to become competitive with GPUs, as shown by the following example.

FPGA vs. GPU for Reduced Precision CNN

Virtex-7 FPGA

- 3,600 DSP48 units
- 433,200 LUTs
- 100–200 MHz

Titan-X GPU

- 3,072 cores (1 ALU per core)
- 1 GHz

- Assume a DSP unit can do one FxP operation.
- One CUDA core can do one FP MAC operation.

- FPGA and GPU have comparable concurrency, but frequency of FPGA is low.

⇒ FPGA throughput much lower than GPU throughput for reduced precision NNs.

FPGA vs. GPU for BNN

- Assume: a CUDA core can perform 32 bitwise operations in each cycle.

⇒ Titan-X GPU can provide ~98,000-fold (=3,072×32) parallelism.

- On FPGA, bitwise operations can be executed even using LUTs.
- Assume: a 6-input LUT can perform 2.5 XNOR operations.

⇒ Virtex-7 FPGA provides ~1,000,000-fold (=2.5×433,000) parallelism.

- Accounting for frequency difference, FPGA and GPU can provide similar throughput for the BNN.

Challenges of BNN

A BNN may use 32-b FP values for first and last layers

⇒ both binary and 32-b FP values need to be stored/processed

⇒ increases complexity

Reference

- S. Mittal, “A survey of FPGA-based accelerators for convolutional neural networks,” *Neural Computing and Applications*, vol. 32, pp. 1109–1139, 2020.